

Human review packet: GHW01DigitalGoods

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GHW01DigitalGoods

Paper id: GHW01DigitalGoods

Source version: GHW01 byte-pinned source inventory

Source record: audit/paper_statement_map.json

Rows: 49

Generated: 2026-08-18

Source URL: [official source](#)

Review status: 0/49 source-claim screens; 0/37 library prerequisite screens. This is a review worksheet while those direct checks remain pending; it is not a final validation receipt.

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GHW01DigitalGoods --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's reviewer-owned review record.

Contents

- [Semantic library prerequisites \(37; not paper claims\)](#)
 - [EconCSLib.Auction.DeterministicOfferBidIndependent](#)
 - [EconCSLib.Auction.DigitalGoodsAuction](#)
 - [EconCSLib.Auction.DigitalGoodsAuction.allocation](#)
 - [EconCSLib.Auction.DigitalGoodsAuction.BinaryAllocation](#)
 - [EconCSLib.Auction.DigitalGoodsAuction.payment](#)
 - [EconCSLib.Auction.DigitalGoodsAuction.utility](#)
 - [EconCSLib.Auction.DigitalGoodsAuction.IndividuallyRational](#)
 - [EconCSLib.Auction.DigitalGoodsAuction.NoPositiveTransfers](#)
 - [EconCSLib.Auction.DigitalGoodsAuction.mk](#)
 - [EconCSLib.Auction.GhwTightLowerAgent](#)
 - [EconCSLib.Auction.GhwTightAgent](#)
 - [EconCSLib.Auction.TwoValueAgent](#)
 - [EconCSLib.Auction.saleCount](#)
 - [EconCSLib.Auction.singlePriceRevenue](#)
 - [EconCSLib.Auction.candidateFixedPriceRevenue](#)
 - [EconCSLib.Auction.deterministicAuctionOffer](#)
 - [EconCSLib.Auction.finiteCandidateFixedPriceBenchmark](#)
 - [EconCSLib.Auction.exists_candidateFixedPriceRevenue_eq_finiteCandidateFixedPriceBenchmark](#)
 - [EconCSLib.Auction.finiteCandidateBenchmarkBidder](#)
 - [EconCSLib.Auction.finiteCandidateOfferPrice](#)
 - [EconCSLib.Auction.finiteCandidatePriceSet](#)
 - [EconCSLib.Auction.ghwTightValue](#)
 - [EconCSLib.pmfExp](#)
 - [EconCSLib.Auction.paper_theorem8_2_raw_cdf_expected_revenue](#)
 - [EconCSLib.Auction.restrictBidsBySide](#)
 - [EconCSLib.Auction.thresholdPriceAuction](#)
 - [EconCSLib.Auction.totalBidValue](#)

- EconCSLib.Auction.twoValueBidProfile
- EconCSLib.Auction.twoValueErasedBidList
- EconCSLib.Auction.twoValueFixedPriceBenchmark
- EconCSLib.Auction.weightedPairingExpectedPayment
- EconCSLib.Auction.weightedPairingExpectedRevenue
- EconCSLib.FairCoin.fairMeasure
- EconCSLib.FairCoin.productMeasure
- EconCSLib.pmfPi
- EconCSLib.pmfProb
- EconCSLib.uniformPMF
- Source claims (49)
 - definition_revenueSpec
 - definition_truthfulnessSpec
 - definition_fixed_price_benchmarkSpec
 - definition_total_valueSpec
 - definition_weighted_pairing_revenueSpec
 - result_theorem4_1Spec
 - result_corollary4_2Spec
 - result_lemma6_1_fixed_size_lower_tailSpec
 - result_theorem6_2_random_samplingSpec
 - result_theorem7_1_weighted_pairingSpec
 - result_theorem7_2_lower_boundSpec
 - result_theorem7_2_tightnessSpec
 - result_lemma8_1_monotone_allocationSpec
 - result_theorem8_2_journal_revenue_upper_boundSpec
 - result_theorem9_1_bid_independent_lower_boundSpec
 - result_lemma9_2_bid_independenceSpec
 - result_theorem9_3_deterministic_lower_boundSpec
 - definition_bounded_supply_fixed_price_benchmarkSpec
 - definition_bounded_supply_top_k_totalSpec
 - definition_bounded_supply_optimal_thresholdSpec
 - model_bounded_single_price_sampling_auctionSpec
 - model_bounded_dual_price_sampling_auctionSpec
 - definition_price_classesSpec
 - definition_auction_correctnessSpec
 - model_unlimited_supplySpec
 - definition_fixed_pricingSpec
 - definition_bidder_countSpec
 - definition_bid_extremaSpec
 - model_bidder_values_and_bidsSpec
 - model_sorted_bid_conventionSpec
 - definition_auction_outcomeSpec
 - definition_auction_mechanismsSpec
 - definition_bidder_profitSpec
 - definition_performance_conventionSpec
 - definition_scale_conditionsSpec
 - definition_competitive_auctionSpec
 - definition_k_item_vickreySpec
 - definition_unlimited_vickrey_familySpec
 - definition_optimal_thresholdSpec
 - definition_bid_independent_auctionSpec
 - definition_threshold_boundary_conventionSpec
 - definition_random_sampling_auctionSpec
 - definition_dual_price_sampling_auctionSpec
 - definition_exact_half_optimal_thresholdSpec
 - definition_independent_half_analysis_modelSpec
 - definition_weighted_pairing_auctionSpec
 - definition_deterministic_optimal_threshold_auctionSpec
 - model_bounded_supply_capacitySpec
 - definition_scarce_supplySpec

Material library prerequisites

EconCSLib.Auction.DeterministicOfferBidIndependent

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ (offer : ℝ → Option ℝ),
  (∀ (report : ℝ), offer report = none) ∧
    ∃ criticalPrice,
      (∀ (report price : ℝ), offer report = some price → price = criticalPrice) ∧
        (∀ report < criticalPrice, offer report = none) ∧
          ∀ (report : ℝ), criticalPrice < report → offer report = some criticalPrice
```

Exact Lean library declaration

```
def DeterministicOfferBidIndependent (offer : ℝ → Option ℝ) : Prop :=
  (∀ report, offer report = none) ∧
    ∃ criticalPrice,
      (∀ report price, offer report = some price → price = criticalPrice) ∧
        (∀ report, report < criticalPrice → offer report = none) ∧
          (∀ report, criticalPrice < report → offer report = some criticalPrice)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.DigitalGoodsAuction

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

Type $u_1 \rightarrow \text{Type } u_1$

Exact Lean library declaration

```
structure DigitalGoodsAuction (Agent : Type*) where
  allocation : (Agent → ℝ) → Agent → ℝ
  payment : (Agent → ℝ) → Agent → ℝ
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.DigitalGoodsAuction.allocation

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
(self : EconCSLib.Auction.DigitalGoodsAuction Agent) → (a : Agent → ℝ) → (a_1 : Agent) → self.1 a a_1
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.DigitalGoodsAuction.BinaryAllocation

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} (M : EconCSLib.Auction.DigitalGoodsAuction Agent) (bids : Agent → ℝ) (i : Agent),  
  M.allocation bids i = 0 ∨ M.allocation bids i = 1
```

Exact Lean library declaration

```
def BinaryAllocation (M : DigitalGoodsAuction Agent) : Prop :=  
  ∀ bids i, M.allocation bids i = 0 ∨ M.allocation bids i = 1
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.DigitalGoodsAuction.payment

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
(self : EconCSLib.Auction.DigitalGoodsAuction Agent) → (a : Agent → ℝ) → (a_1 : Agent) → self.2 a a_1
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.DigitalGoodsAuction.utility

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
  (M : EconCSLib.Auction.DigitalGoodsAuction Agent) →  
    (values : Agent → ℝ) → (i : Agent) → (bids : Agent → ℝ) → values i * M.allocation bids i - M.payment bids i
```

Exact Lean library declaration

```
def utility (M : DigitalGoodsAuction Agent)  
  (values : Agent → ℝ) (i : Agent) (bids : Agent → ℝ) : ℝ :=  
  values i * M.allocation bids i - M.payment bids i
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.DigitalGoodsAuction.IndividuallyRational

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\forall \{ \text{Agent} : \text{Type } u_1 \} (M : \text{EconCSLib.Auction.DigitalGoodsAuction Agent}) (\text{values} : \text{Agent} \rightarrow \mathbb{R}) (i : \text{Agent}),$
 $\quad \quad \quad 0 \leq M.\text{utility } \text{values } i \text{ values}$

Exact Lean library declaration

`def IndividuallyRational (M : DigitalGoodsAuction Agent) : Prop :=`
 `$\forall (\text{values} : \text{Agent} \rightarrow \mathbb{R}) (i : \text{Agent}), 0 \leq M.\text{utility } \text{values } i \text{ values}$`

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.DigitalGoodsAuction.NoPositiveTransfers

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\forall \{ \text{Agent} : \text{Type } u_1 \} (M : \text{EconCSLib.Auction.DigitalGoodsAuction Agent}) (\text{bids} : \text{Agent} \rightarrow \mathbb{R}) (i : \text{Agent}),$
 $\theta \leq M.\text{payment bids } i$

Exact Lean library declaration

def NoPositiveTransfers (M : DigitalGoodsAuction Agent) : Prop :=
 $\forall (\text{bids} : \text{Agent} \rightarrow \mathbb{R}) (i : \text{Agent}), \theta \leq M.\text{payment bids } i$

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.DigitalGoodsAuction.mk

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

{Agent : Type u_1} → ((Agent → ℝ) → Agent → ℝ) → ((Agent → ℝ) → Agent → ℝ) → EconCSLib.Auction.DigitalGoodsAuction Agent

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.GhwTightLowerAgent

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$(k : \mathbb{N}) \rightarrow (r : \text{Fin } (k - 1)) \times \text{Fin } (2^k - (r + 1))$

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.GhwTightAgent

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$(k \ s : \mathbb{N}) \rightarrow \text{EconCSLib.Auction.GhwTightLowerAgent } k \ \oplus \text{Fin } s$

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.TwoValueAgent

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

(highCount lowCount : ℕ) → Fin highCount * Fin lowCount

Exact Lean library declaration

```
abbrev TwoValueAgent (highCount lowCount : ℕ) :=  
  Fin highCount * Fin lowCount
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.saleCount

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} → [inst : Fintype Agent] → (values : Agent → ℝ) → (p : ℝ) → {i | p ≤ values i}.card
```

Exact Lean library declaration

```
noncomputable def saleCount [Fintype Agent] (values : Agent → ℝ) (p : ℝ) : ℕ :=  
  ((Finset.univ : Finset Agent).filter fun i => p ≤ values i).card
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.singlePriceRevenue

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\{ \text{Agent} : \text{Type } u_1 \} \rightarrow [\text{inst} : \text{Fintype Agent}] \rightarrow (\text{values} : \text{Agent} \rightarrow \mathbb{R}) \rightarrow (p : \mathbb{R}) \rightarrow \sum i, \text{if } p \leq \text{values } i \text{ then } p \text{ else } 0$

Exact Lean library declaration

```
noncomputable def singlePriceRevenue [Fintype Agent]
  (values : Agent → ℝ) (p : ℝ) : ℝ :=
  ∑ i : Agent, if p ≤ values i then p else 0
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.candidateFixedPriceRevenue

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
  [inst : Fintype Agent] →  
    (values : Agent → ℝ) →  
      (minWinners : ℕ) →  
        (i : Agent) →  
          if 0 ≤ values i ∧ minWinners ≤ EconCSLib.Auction.saleCount values (values i) then  
            EconCSLib.Auction.singlePriceRevenue values (values i)  
          else 0
```

Exact Lean library declaration

```
noncomputable def candidateFixedPriceRevenue [Fintype Agent]  
  (values : Agent → ℝ) (minWinners : ℕ) (i : Agent) : ℝ :=  
  if 0 ≤ values i ∧ minWinners ≤ saleCount values (values i) then  
    singlePriceRevenue values (values i)  
  else 0
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.deterministicAuctionOffer

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
[inst : DecidableEq Agent] →
(M : EconCSLib.Auction.DigitalGoodsAuction Agent) →
(bids : Agent → ℝ) →
(i : Agent) →
(a : ℝ) →
  have profile := Function.update bids i a;
  if M.allocation profile i = 1 then some (M.payment profile i) else none
```

Exact Lean library declaration

```
noncomputable def deterministicAuctionOffer [DecidableEq Agent]
  (M : DigitalGoodsAuction Agent) (bids : Agent → ℝ) (i : Agent) :
  ℝ → Option ℝ :=
fun report =>
  let profile := Function.update bids i report
  if M.allocation profile i = 1 then
    some (M.payment profile i)
  else
    none
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.finiteCandidateFixedPriceBenchmark

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
[inst : Fintype Agent] →
[inst_1 : Nonempty Agent] →
(values : Agent → ℝ) →
(minWinners : ℕ) → Finset.univ.sup' ... (EconCSLib.Auction.candidateFixedPriceRevenue values minWinners)
```

Exact Lean library declaration

```
noncomputable def finiteCandidateFixedPriceBenchmark [Fintype Agent]
[Nonempty Agent] (values : Agent → ℝ) (minWinners : ℕ) : ℝ :=
(Finset.univ : Finset Agent).sup'
  (by
    obtain {i} := (inferInstance : Nonempty Agent)
    exact {i, by simp})
(candidateFixedPriceRevenue values minWinners)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.exists_candidateFixedPriceRevenue_eq_finiteCandidateFixedPriceBenchmark

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : Nonempty Agent] (values : Agent → ℝ) (minWinners : ℕ),
  ∃ i,
    EconCSLib.Auction.finiteCandidateFixedPriceBenchmark values minWinners =
      EconCSLib.Auction.candidateFixedPriceRevenue values minWinners i
```

Exact Lean library declaration

```
theorem exists_candidateFixedPriceRevenue_eq_finiteCandidateFixedPriceBenchmark
  [Fintype Agent] [Nonempty Agent]
  (values : Agent → ℝ) (minWinners : ℕ) :
  ∃ i : Agent,
    finiteCandidateFixedPriceBenchmark values minWinners =
      candidateFixedPriceRevenue values minWinners i
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.finiteCandidateBenchmarkBidder

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
  [inst : Fintype Agent] → [inst_1 : Nonempty Agent] → (values : Agent → ℝ) → (minWinners : ℕ) → Classical.choose ...
```

Exact Lean library declaration

```
noncomputable def finiteCandidateBenchmarkBidder [Fintype Agent]  
  [Nonempty Agent] (values : Agent → ℝ) (minWinners : ℕ) : Agent :=  
  Classical.choose  
    (exists_candidateFixedPriceRevenue_eq_finiteCandidateFixedPriceBenchmark  
      values minWinners)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.finiteCandidateOfferPrice

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
  [inst : Fintype Agent] →
    [inst_1 : Nonempty Agent] →
      (values : Agent → ℝ) →
        (minWinners : ℕ) →
          have i := EconCSLib.Auction.finiteCandidateBenchmarkBidder values minWinners;
          if 0 ≤ values i ∧ minWinners ≤ EconCSLib.Auction.saleCount values (values i) then values i else 0
```

Exact Lean library declaration

```
noncomputable def finiteCandidateOfferPrice [Fintype Agent]
  [Nonempty Agent] (values : Agent → ℝ) (minWinners : ℕ) : ℝ :=
  let i := finiteCandidateBenchmarkBidder values minWinners
  if 0 ≤ values i ∧ minWinners ≤ saleCount values (values i) then
    values i
  else
    0
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.finiteCandidatePriceSet

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} → [inst : Fintype Agent] → (values : Agent → ℝ) → insert 0 (Finset.image values Finset.univ)
```

Exact Lean library declaration

```
noncomputable def finiteCandidatePriceSet [Fintype Agent]  
  (values : Agent → ℝ) : Finset ℝ :=  
  insert 0 ((Finset.univ : Finset Agent).image values)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.ghwTightValue

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(k s : ℕ) →  
(a : EconCSLib.Auction.GhwTightAgent k s) →  
  match a with  
  | Sum.inl a => 2 ^ (↑a.fst + 1)  
  | Sum.inr val => 2 ^ k
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.pmfExp

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\{\alpha : \text{Type } u_1\} \rightarrow [\text{inst} : \text{Fintype } \alpha] \rightarrow [\text{DecidableEq } \alpha] \rightarrow (\mu : \text{PMF } \alpha) \rightarrow (f : \alpha \rightarrow \mathbb{R}) \rightarrow \sum a, (\mu a).toReal * f a$

Exact Lean library declaration

```
noncomputable def pmfExp {α : Type*} [Fintype α] [DecidableEq α]
  (μ : PMF α) (f : α → ℝ) : ℝ :=
  ∑ a, (μ a).toReal * f a
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.paper_theorem8_2_raw_cdf_expected_revenue

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Price : Type u_2} →
[inst : Fintype Agent] →
[inst_1 : Fintype Price] →
[inst_2 : DecidableEq Price] →
(values : Agent → ℝ) →
(price : Price → ℝ) →
(offerLaw : Agent → PMF Price) →
  ∑ i, EconCSLib.pmfExp (offerLaw i) fun p => if price p ≤ values i then price p else 0
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.restrictBidsBySide

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
  (side : Agent → Bool) → (keep : Bool) → (bids : Agent → ℝ) → (a : Agent) → if side a = keep then bids a else 0
```

Exact Lean library declaration

```
def restrictBidsBySide (side : Agent → Bool) (keep : Bool)  
  (bids : Agent → ℝ) : Agent → ℝ :=  
  fun j => if side j = keep then bids j else 0
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.thresholdPriceAuction

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
[DecidableEq Agent] →  
  (threshold : (Agent → ℝ) → Agent → ℝ) →  
    { allocation := fun bids i => if threshold bids i ≤ bids i then 1 else 0,  
      payment := fun bids i => if threshold bids i ≤ bids i then threshold bids i else 0 }
```

Exact Lean library declaration

```
noncomputable def thresholdPriceAuction [DecidableEq Agent]  
  (threshold : (Agent → ℝ) → Agent → ℝ) : DigitalGoodsAuction Agent where  
  allocation bids i := if threshold bids i ≤ bids i then 1 else 0  
  payment bids i := if threshold bids i ≤ bids i then threshold bids i else 0
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.totalBidValue

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} → [inst : Fintype Agent] → (values : Agent → ℝ) → ∑ i, values i
```

Exact Lean library declaration

```
noncomputable def totalBidValue [Fintype Agent] (values : Agent → ℝ) : ℝ :=  
  ∑ i : Agent, values i
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.twoValueBidProfile

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(H highCount lowCount : ℕ) →  
(a : EconCSLib.Auction.TwoValueAgent highCount lowCount) →  
  match a with  
  | Sum.inl val => ↑H  
  | Sum.inr val => 1
```

Exact Lean library declaration

```
def twoValueBidProfile (H highCount lowCount : ℕ) :  
  TwoValueAgent highCount lowCount → ℝ  
  | Sum.inl _ => (H : ℝ)  
  | Sum.inr _ => 1  
  
theorem singlePriceRevenue_twoValueBidProfile_one  
  {H highCount lowCount : ℕ}  
  (hH_ge_one : 1 ≤ (H : ℝ)) :  
  singlePriceRevenue (twoValueBidProfile H highCount lowCount) 1 =  
    (highCount + lowCount : ℝ) := by  
  classical  
  unfold singlePriceRevenue  
  simp [twoValueBidProfile, hH_ge_one, Finset.sum_const, nsmul_eq_mul]
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.twoValueErasedBidList

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(H highCount lowCount : ℕ) → List.replicate highCount ↑H ++ List.replicate lowCount 1
```

Exact Lean library declaration

```
def twoValueErasedBidList (H highCount lowCount : ℕ) : List ℝ :=  
  List.replicate highCount (H : ℝ) ++ List.replicate lowCount 1
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.twoValueFixedPriceBenchmark

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(H highCount lowCount : ℕ) →
  if hzero : highCount + lowCount = 0 then 0
  else
    EconCSLib.Auction.finiteCandidateFixedPriceBenchmark (EconCSLib.Auction.twoValueBidProfile H highCount lowCount) 1
```

Exact Lean library declaration

```
noncomputable def twoValueFixedPriceBenchmark
  (H highCount lowCount : ℕ) : ℝ :=
  if hzero : highCount + lowCount = 0 then 0 else
    @finiteCandidateFixedPriceBenchmark
      (TwoValueAgent highCount lowCount) inferInstance
      (by
        have hsum_pos : 0 < highCount + lowCount :=
          Nat.pos_of_ne_zero hzero
        by_cases hhigh_pos : 0 < highCount
        · exact (Sum.inl (0, hhigh_pos))
        · have hhigh_zero : highCount = 0 :=
            Nat.eq_zero_of_not_pos hhigh_pos
          have hlow_pos : 0 < lowCount := by
            by_contra hlow_not
            have hlow_zero : lowCount = 0 :=
              Nat.eq_zero_of_not_pos hlow_not
            simp [hhigh_zero, hlow_zero] at hsum_pos
            exact (Sum.inr (0, hlow_pos)))
      (twoValueBidProfile H highCount lowCount) 1
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.weightedPairingExpectedPayment

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
  [inst : Fintype Agent] →
    [inst_1 : DecidableEq Agent] →
      (values : Agent → ℝ) →
        (i : Agent) →
          ∑ j,
            if j ≠ i ∧ values j ≤ values i then values j ^ 2 / (EconCSLib.Auction.totalBidValue values - values i)
            else 0
```

Exact Lean library declaration

```
noncomputable def weightedPairingExpectedPayment
  [Fintype Agent] [DecidableEq Agent]
  (values : Agent → ℝ) (i : Agent) : ℝ :=
  ∑ j : Agent,
    if j ≠ i ∧ values j ≤ values i then
      (values j) ^ 2 / (totalBidValue values - values i)
    else
      0
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Auction.weightedPairingExpectedRevenue

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
  [inst : Fintype Agent] →  
    [inst_1 : DecidableEq Agent] → (values : Agent → ℝ) → ∑ i, EconCSLib.Auction.weightedPairingExpectedPayment values i
```

Exact Lean library declaration

```
noncomputable def weightedPairingExpectedRevenue  
  [Fintype Agent] [DecidableEq Agent]  
  (values : Agent → ℝ) : ℝ :=  
  ∑ i : Agent, weightedPairingExpectedPayment values i
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairCoin.fairMeasure

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(PMF.bernoulli (1 / 2) EconCSLib.FairCoin.fairMeasure._proof_2).toMeasure
```

Exact Lean library declaration

```
noncomputable def fairMeasure : Measure Bool :=  
  ((PMF.bernoulli (1 / 2 : NNReal) (by norm_num)).toMeasure : Measure Bool)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairCoin.productMeasure

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(ι : Type u_1) → MeasureTheory.Measure.infinitePi fun x => EconCSLib.FairCoin.fairMeasure
```

Exact Lean library declaration

```
noncomputable def productMeasure (ι : Type*) : Measure (ι → Bool) :=  
  Measure.infinitePi (fun _ : ι => fairMeasure)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.pmfPi

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{ι : Type u_1} →  
  [inst : Fintype ι] →  
    [inst_1 : DecidableEq ι] →  
      {α : ι → Type u_2} →  
        [inst_2 : (i : ι) → Fintype (α i)] → (μ : (i : ι) → PMF (α i)) → PMF.ofFintype (fun f => ∏ i, (μ i) (f i)) ...
```

Exact Lean library declaration

```
noncomputable def pmfPi {ι : Type*} [Fintype ι] [DecidableEq ι]  
  {α : ι → Type*} [∀ i : ι, Fintype (α i)]  
  (μ : ∀ i : ι, PMF (α i)) : PMF ((i : ι) → α i) :=  
  PMF.ofFintype (fun f : (i : ι) → α i => ∏ i : ι, μ i (f i)) (by  
    classical  
    have hcoord : ∀ i : ι, ∑ a : α i, μ i a = 1 := by  
      intro i  
      rw [← PMF.tsum_coe (μ i), tsum_fintype]  
    calc  
      ∑ f : ((i : ι) → α i), ∏ i : ι, μ i (f i)  
      = ∑ f ∈ Fintype.piFinset  
        (fun i : ι => (Finset.univ : Finset (α i))),  
        ∏ i : ι, μ i (f i) := by  
        simp  
      _ = ∏ i : ι, ∑ a ∈ (Finset.univ : Finset (α i)), μ i a := by  
        symm  
        exact Finset.prod_univ_sum  
          (t := fun i : ι => (Finset.univ : Finset (α i)))  
          (f := fun i a => μ i a)  
      _ = 1 := by simp [hcoord])
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.pmfProb

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →  
  [inst : Fintype α] →  
    [inst_1 : DecidableEq α] →  
      (μ : PMF α) → (p : α → Prop) → [inst_2 : DecidablePred p] → EconCSLib.pmfExp μ fun a => if p a then 1 else 0
```

Exact Lean library declaration

```
noncomputable def pmfProb {α : Type*} [Fintype α] [DecidableEq α]  
  (μ : PMF α) (p : α → Prop) [DecidablePred p] : ℝ :=  
  pmfExp μ (fun a => if p a then 1 else 0)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.uniformPMF

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$(\alpha : \text{Type } u_1) \rightarrow [\text{inst} : \text{Fintype } \alpha] \rightarrow [\text{inst}_1 : \text{Nonempty } \alpha] \rightarrow \text{PMF.ofFintype } (\text{fun } x \Rightarrow (\uparrow(\text{Fintype.card } \alpha))^{-1}) \dots$

Exact Lean library declaration

```
noncomputable def uniformPMF (α : Type*) [Fintype α] [Nonempty α] : PMF α :=
  PMF.ofFintype (fun _ : α => ((Fintype.card α : ENNReal)⁻¹)) (by
    rw [Finset.sum_const, Finset.card_univ]
    have hcard : (Fintype.card α : ENNReal) ≠ 0 := by
      exact_mod_cast (Fintype.card_pos_iff.mpr <Nonempty α>.ne')
    simp [nsmul_eq_mul] using
      (ENNReal.mul_inv_cancel hcard (ENNReal.natCast_ne_top (Fintype.card α))))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 211-214

Verbatim source input

outcomes. We use R to denote the auction revenue for a particular auction mechanism and set of bids. R is the sum of all sale prices. For randomized auctions, R is a random variable.

Expanded PaperInterface specification

```
(∀ {Agent : Type u_1} (outcome : GHW01DigitalGoods.ProofBridge.source_auctionOutcome Agent),
  GHW01DigitalGoods.ProofBridge.source_outcomeRevenue outcome =
  GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.revenue outcome) ∧
(∀ {Agent : Type u_2} (auction : GHW01DigitalGoods.ProofBridge.source_deterministicAuction Agent)
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
  GHW01DigitalGoods.ProofBridge.source_deterministicRevenue auction bids =
  GHW01DigitalGoods.SourceDefinitions.deterministicRevenue auction bids) ∧
∀ {Agent : Type u_3} (auction : GHW01DigitalGoods.ProofBridge.source_randomizedAuction Agent)
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
  GHW01DigitalGoods.ProofBridge.source_randomizedRevenueLaw auction bids =
  GHW01DigitalGoods.SourceDefinitions.randomizedRevenueLaw auction bids
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_revenue_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 215-222

Verbatim source input

We say that an auction is truthful if bidding u_i is a dominant strategy for bidder i . More specifically, let a bidder's profit be the difference between the bidder's utility value and the price the bidder pays if they win the auction, or zero if they lose. Then an auction is truthful if a bidder's profit (or expected profit, for randomized auctions), as a function of the bidder's bid, is maximized at the bidder's utility value, for any fixed values of the other bidders' bids. Truthful

Expanded PaperInterface specification

```
(∀ {Agent : Type u_1} (auction : GHW01DigitalGoods.ProofBridge.source_deterministicAuction Agent),
  GHW01DigitalGoods.ProofBridge.source_deterministicTruthful auction =
    GHW01DigitalGoods.SourceDefinitions.DeterministicTruthful auction) ∧
  (∀ {Agent : Type u_2} (auction : GHW01DigitalGoods.ProofBridge.source_randomizedAuction Agent),
    GHW01DigitalGoods.ProofBridge.source_randomizedTruthful auction =
      GHW01DigitalGoods.SourceDefinitions.RandomizedTruthful auction)
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_truthfulness_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 134-138

Verbatim source input

that can be obtained from this set of bidders. Let F be the optimal fixed pricing revenue. Clearly $F \leq T$. F is an upper bound on the revenue that can be obtained by any fixed priced sale or any single-price auction. We want revenues of our truthful auctions

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : Nonempty Agent]
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
  GHW01DigitalGoods.ProofBridge.source_optimalFixedPriceRevenue bids =
    GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue bids
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_fixed_price_benchmark_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 132-134

Verbatim source input

h. Let the total utility T be the sum of all bidders' utilities. T is an obvious upper bound on the revenue that can be obtained from this set of bidders. Let F

[Next verbatim source excerpt]

does not discourage bidders from bidding their utility value. When considering truthful auctions, we assume that $b_i = u_i$. To enable analysis of auction revenue we define several properties of an input set of bids. As stated in the introduction, T is the sum of all of the bids. An equivalent definition of T is the revenue due to the optimal multi-price untruthful auction, the one that satisfies all bids at their value. The revenue for optimal fixed pricing is F . Note that F can also be

Expanded PaperInterface specification

```
(∀ {Agent : Type u_1} [inst : Fintype Agent] (values : Agent → ℝ),
  GHW01DigitalGoods.ProofBridge.totalValue values = ∑ i, values i) ∧
  ∀ {Agent : Type u_2} [inst : Fintype Agent] (input : GHW01DigitalGoods.ProofBridge.source_auctionInput Agent),
    GHW01DigitalGoods.SourceDefinitions.TruthfulReports input →
      GHW01DigitalGoods.ProofBridge.totalValue input.values =
        GHW01DigitalGoods.SourceDefinitions.totalBidValue input.bids ∧
        GHW01DigitalGoods.SourceDefinitions.totalBidValue input.bids =
          (GHW01DigitalGoods.SourceDefinitions.fullValueOutcome input.bids).revenue
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_total_value_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 621-623

Verbatim source input

We use the following notation. We denote the expected revenue of the $\sqrt{}$ wighted pairing auction by W ,
let $k = \log h$ and $s = \log h$ First we give bound the

Expanded PaperInterface specification

```
(∀ {Agent : Type u_1} [inst : Fintype Agent] (bids : Agent → ℝ) (hbid_ge_one : ∀ (i : Agent), 1 ≤ bids i)
  (hdistinct : ∀ (i : Agent), ∃ j, j ≠ i),
  GHW01DigitalGoods.ProofBridge.source_weightedPairingW bids hbid_ge_one hdistinct =
  GHW01DigitalGoods.SourceDefinitions.weightedPairingW bids ...) ∧
(∀ {Agent : Type u_2} [inst : Fintype Agent] (bids : Agent → ℝ) (hbid_ge_one : ∀ (i : Agent), 1 ≤ bids i)
  (hdistinct : ∀ (i : Agent), ∃ j, j ≠ i),
  GHW01DigitalGoods.ProofBridge.source_weightedPairingOutcomeExpectedRevenue bids hbid_ge_one hdistinct =
  GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeExpectedRevenue bids ...) ∧
∀ {Agent : Type u_3} [inst : Fintype Agent] (bids : Agent → ℝ) (hbid_ge_one : ∀ (i : Agent), 1 ≤ bids i)
  (hdistinct : ∀ (i : Agent), ∃ j, j ≠ i),
  GHW01DigitalGoods.ProofBridge.source_weightedPairingOutcomeExpectedRevenue bids hbid_ge_one hdistinct =
  GHW01DigitalGoods.ProofBridge.source_weightedPairingW bids hbid_ge_one hdistinct
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_weighted_pairing_revenue_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 359-370

Verbatim source input

Theorem 4.1 $F \geq T / (2 \log h)$.
Proof. Divide the bids into $\log h$ bins by partitioning the bids at values of powers of two. Thus, each bid is less than a factor of two from any other bid in the same bin. Since the sum of the bids is T and there are $\log h$ bins, then some bin has a sum of at least $T / \log h$. Note that the lowest bid in this bin is at least half of any other bid in the bin. If the optimal single-price auction chose, as its selling price, the price of the lowest bid in this bin, then the contribution of each bid in this bin to the revenue is at least half of the bid's value. Since the bin sums to more than $T / \log h$, this means that the revenue is greater than $T / (2 \log h)$. Thus the optimal fixed pricing can always achieve a revenue of at least $T / (2 \log h)$.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

Let $h \geq 2$. For a nonempty finite bid profile whose values all lie in $[1, h]$, its total value T satisfies $T \leq (2 \log_2 h) F$,
 $\hookrightarrow$ where F is the optimal fixed-price revenue.

Archival source anchor: cited publication, lines 359-370

Recorded basis User instruction 2026-07-25: assume whatever convention makes the result true, provided the convention is explicit and the archival claim is not represented as proved.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (values : Agent → ℝ) {h : ℝ},
  2 ≤ h →
  (∀ (i : Agent), 1 ≤ values i) →
  (∀ (i : Agent), values i ≤ h) →
  ∀ (_highest : ∃ i, values i = h),
  have x := ...;
  GHW01DigitalGoods.ProofBridge.totalValue values ≤
  2 * Real.logb 2 h * GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem4_1

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 301-307

Verbatim source input

Corollary 4.2 $F \geq T / (4 \log n)$.
Proof. Let p be the optimal price; clearly $p \geq h/n$.
If one drops all bids with values below p , F does not change and T decreases by at most a factor of two.
After the bids are dropped, the ratio of the highest and the lowest bid values is at most n , yielding the desired result.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

Let $n \geq 2$ be the number of bidders. For a nonempty finite nonnegative bid profile with positive maximum value h and every value $\hookrightarrow$ at most h , $T \leq (4 \log_2 n) F$, where F is the optimal fixed-price revenue.

Archival source anchor: cited publication, lines 301-307

Recorded basis User instruction 2026-07-25: assume whatever convention makes the result true, provided the convention is explicit and the archival claim is not represented as proved.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (values : Agent → ℝ) {h : ℝ},
  (∀ (i : Agent), 0 ≤ values i) →
    0 < h →
      ∀ (hmax : ∃ i, values i = h),
        (∀ (i : Agent), values i ≤ h) →
          2 ≤ ↑(Fintype.card Agent) →
            have x := ...;
            GHW01DigitalGoods.ProofBridge.totalValue values ≤
              4 * Real.logb 2 ↑(Fintype.card Agent) * GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_corollary4_2

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 428-454

Verbatim source input

Lemma 6.1 Consider a set A and its subset $B \subset A$. Suppose we pick an integer k such that $0 < k < |A|$ and a random subset (sample) $S \subset A$ of size k . Then for $0 < \delta \leq 1$ we have $\Pr[|S \cap B| < (1 - \delta)|B| \cdot k/|A|] < \exp(-|B| \cdot k \delta^2 / (2|A|))$. Proof. We refer to elements of A as points. Note

Random Sampling Optimal that $|S \cap B|$ is the number of sample points in B , and its expected value is $|B| \cdot k/|A|$. Let $p = k/|A|$. Threshold Auction

If instead of selecting a sample of size exactly k we choose each point to be in the sample independently with probability p then the Chernoff bound would yield the lemma.

Let $A = \{a_1, \dots, a_n\}$ and without loss of generality assume that $B = \{a_1, \dots, a_k\}$. We can view the process of selecting S as follows. Consider the elements of A in the order induced by the indices. For each element a_i considered, select the element with probability p_i , where p_i depends on the selections made up to this point. Let t be the number of points already selected when a_{i+1} is considered. Then $i - t$ is the number of points considered but not selected. Suppose that $t/i < p$. Then $p_{i+1} > p$. We conclude that when we select the sample as a random subset of size k , the probability that the number of sample points in B is less than the expected

Expanded PaperInterface specification

```
∀ {α : Type u_1} {A B : Finset α},
  B ⊆ A →
  ∀ {k : ℕ},
    0 < k →
    k < A.card →
    ∀ {δ : ℝ},
      0 < δ →
      δ ≤ 1 →
      (GHW01DigitalGoods.fixedSizeSampleProbability A k fun sample =>
        ↑(GHW01DigitalGoods.fixedSizeHitCount B sample) < (1 - δ) * ↑B.card * ↑k / ↑A.card) <
        Real.exp (-(↑B.card * ↑k * δ ^ 2 / (2 * ↑A.card)))
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_lemma6_1_fixed_size_lower_tail

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 455-547

Verbatim source input

[No record available.]

Expanded PaperInterface specification

```
∀ (m : ℕ) (_hm_pos : 0 < m),
  have x := ...;
  ∀ (values : Fin (m + m) → ℝ) {alpha : ℕ} {highValue : ℝ},
    0 < highValue →
      (∀ (i : Fin (m + m)), values i ≤ highValue) →
        (∃ i, values i = highValue) →
          ↑alpha * highValue ≤ GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values →
            1 - Real.exp (-↑alpha / 36) - 40 * Real.exp (-↑alpha / 72) ≤
              GHW01DigitalGoods.pmfEventProbability (GHW01DigitalGoods.uniformFixedSizeSampleLaw m ...) fun sample =>
                GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values ≤
                  6 *
                    GHW01DigitalGoods.ProofBridge.revenue
                      (GHW01DigitalGoods.randomSamplingOptimalThresholdAuction
                        (GHW01DigitalGoods.sampledSideAssignment ↑sample) true 1)
                      values
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem6_2_random_sampling

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 563-607

Verbatim source input

Theorem 7.1 If $4h \leq T$, then for the weighted pairing auction $E[R] = \Omega(T / \log h)$.

[U+0002 control character]
[U+0002 control character] S_j
We are interested in $E[R] \geq$
 $E[R] \geq$
 $48T$.

Since S_j 's sum up to T , the sum is minimized when

Proof. Partition the n bids into $\log h$ bins as be- all $\log h$ of the S_j 's are equal to $T / \log h$. In this
fore such that bin j contains only bids in the interval case,
 $[2^{j-1}, 2^j]$. Recall that an important property of these

$T / 2 (\log h)$
bins is that bids in the same bin are within factor
 $E[R] \geq$
 $48T \log_2 h$

of two from each other. Let S_j be the sum of the
bids in bin j . Note that the sum of the contents of

bins
contain only one bid is upper bounded by Since $T > T / 2$, we have
[U+0002 control character] $\log$ that

h_j
 $j=1$ $2 = 2h - 1 < T / 2$. We will ignore such bins
 T

in our analysis below. Consider all bins with two or
 $E[R] \geq$

192
 $\log h$
more bids, and let T be the sum of bids in these

bins. We have $T > T / 2$.
Thus, $E[R] = \Omega(T / \log h)$.
For each bin j , we look at pairings of bids that
are both in j and we bound the expected revenue
Note that the constant here is $1/192$ which does
due to such pairings. First, the probability that a
not seem too good. However, the analysis was very
bid i in bin j is paired with another bid in bin j is
loose in contributions to R that it considered. In
 $(S_j - b_i) / (T - b_i) > S_j / (3T)$, since bin j contains at
practice, this auction has much better constants. See
least two bids and $S_j - b_i \geq S_j / 3$.
Section 10 for details.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

Let $h \geq 2$. Under Theorem 7.1's normalized bid and large-market conditions, the operational weighted-pairing auction satisfies T
 $\hookrightarrow \leq 576 \log_2(h) W$.

Archival source anchor: cited publication, lines 563-607

Recorded basis User instruction 2026-07-25: assume whatever convention makes the result true, provided the convention is explicit
and the archival claim is not represented as proved.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (values : Agent → ℝ) {h total : ℝ},
  total = GHW01DigitalGoods.ProofBridge.totalValue values →
    2 ≤ h →
      (∀ (i : Agent), 1 ≤ values i) →
        (∀ (i : Agent), values i ≤ h) →
          (∃ i, values i = h) →
            4 * h ≤ total → total ≤ 576 * Real.logb 2 h * GHW01DigitalGoods.ProofBridge.weightedPairingRevenue values
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem7_1_weighted_pairing

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 615-680

Verbatim source input

We show that the expected revenue of the weighted pairing auction is $\Omega(F / \log h)$, and that this bound is tight in the worst case. This implies that the weighted pairing auction is not competitive. We use the following notation. We denote the expected revenue of the weighted pairing auction by W , let $k = \log h$ and $s = \log h$. First we give bound the revenue from below.

Theorem 7.2 If $F \geq 2h$, then $W = \Omega(F / \log h)$ and this bound is tight.

$i-1$
 $i-1$
>
.
 $2(k - i) + (i - 1)$
 $2k$
This comes from assuming that all bids below bid i are at value 2^{j-1} and all bids above bid i are at value 2^j . This is the worst it could possibly be. The expected number of bids in bin j that win when paired with other bids from the bin is at least k
[U+0001 control character]
 $i-1$
 $i=1$
 $2k$
=
 $k-1$
.
4
The smallest bid in bin j has the value of at least 2^{j-1} . Proof. If $F \leq T/s$, then since $W = \Omega(T/k)$ we have $S_j / (2k)$. Let R_j be the revenue generated by bids in $W = \Omega(T/s)$. bin j being paired with other bids in bin j . We have: Assume $F > T/s$. Partition the bids over buckets, with bucket i containing bids in the range $[2^i, 2^{i+1})$.
 $S_j S_j k - 1$
Let M be the set of bids in a bucket with the largest
.
.
 $E[R_j] \geq$
 $3T/2k$
4
total bid value and assume that M is defined by the $(k - 1)S_j/2$ value range $[t, 2t)$. We show that the revenue due $\geq$
 $24kT$
only to M is big. The assumption $F \geq 2h$ implies
7
[form-feed in source extraction]
8
that $|M| \geq 2$. (If $|M| = 1$ then $M = \{h\}$ and the previous bucket contains a single bid; in this case $F < 2h$.) Recall that $|M|t = \Omega(F)$. Let M' be the highest $\lfloor |M|/2 \rfloor$ elements of M and let $M'' = M \setminus M'$. The probability that an item in M' is paired up with an item in M'' is $\Omega(T/F) = \Omega(1/s)$, and the expected revenue is $\Omega(|M'|t/s) = \Omega(F/s)$.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

Let $h \geq 2$ and $F \geq 2h$. Under Theorem 7.2's normalized bid conditions, the operational weighted-pairing auction satisfies $F \leq \frac{1}{2304} \sqrt{\log_2 h} W$.

Archival source anchor: cited publication, lines 615-680

Recorded basis User instruction 2026-07-25: assume whatever convention makes the result true, provided the convention is explicit and the archival claim is not represented as proved.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (values : Agent → ℝ) {h total : ℝ},
  total = GHW01DigitalGoods.ProofBridge.totalValue values →
    2 ≤ h →
```

```

(∀ (i : Agent), 1 ≤ values i) →
  (∀ (i : Agent), values i ≤ h) →
    ∀ (_highest : ∃ i, values i = h),
      have x := ...;
      2 * h ≤ GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values →
        GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values ≤
          2304 * √(Real.logb 2 h) * GHW01DigitalGoods.ProofBridge.weightedPairingRevenue values

```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem7_2_lower_bound

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 626-720

Verbatim source input

Theorem 7.2 If $F \geq 2h$, then $W = \Omega(F / \log h)$ and this bound is tight.

```
i-1
i-1
>
.
2(k - i) + (i - 1)
2k
This comes from assuming that all bids below bid i
are at value  $2^{j-1}$  and all bids above bid i are at value
 $2^j$ . This is the worst it could possibly be. The expected number of bids in bin j that win when paired
with other bids from the bin is at least
k
[U+0001 control character]
i-1
i=1

2k

=

k-1
.
4
The smallest bid in bin j has the value of at least Proof. If  $F \leq T/s$ , then since  $W = \Omega(T/k)$  we have
 $S_j/(2k)$ . Let  $R_j$  be the revenue generated by bids in  $W = \Omega(T/s)$ .
bin j being paired with other bids in bin j. We have:
Assume  $F > T/s$ . Partition the bids over buckets,
with bucket i containing bids in the range  $[2^i, 2^{i+1})$ .
 $S_j S_j k - 1$ 
Let M be the set of bids in a bucket with the largest
.
.
 $E[R_j] \geq$ 
 $3T/2k$ 
4
total bid value and assume that M is defined by the
 $(k - 1)S_j/2$ 
value range  $[t, 2t)$ . We show that the revenue due
 $\geq$ 
 $24kT$ 
only to M is big. The assumption  $F \geq 2h$  implies
7

[form-feed in source extraction]
8
that  $|M| \geq 2$ . (If  $|M| = 1$  then  $M = \{h\}$  and the
previous bucket contains a single bid; in this case
 $F < 2h$ .) Recall that  $|M|t = \Omega(F)$ . Let  $M'$  be the
highest  $\lfloor |M|/2 \rfloor$  elements of M and let  $M'' = M \setminus M'$ .
The probability that an item in  $M'$  is paired up
with an item in  $M''$  is  $\Omega(T/F) = \Omega(1/s)$ , and the
expected revenue is  $\Omega(|M'|t/s) = \Omega(F/s)$ .
The above argument considers only contributions
of items in M. Somewhat surprisingly, the resulting
bound is tight up to a constant factor. We give an example where the expected revenue W of the weighted
pairing auction is  $\Omega(T/s)$ . The bid values are  $2^i$  for
 $i = 1, \dots, k$ . For each  $i = 1, \dots, k - 1$  the number
of bids of this value is  $2^{k-i}$  and the total value of
such bids is  $2^k$ . For the last value,  $2^k$ , there are s
bids of this value for the total of  $s2^k = F$ . Note that
 $T = 2^k (s + k - 1)$ .
We show that  $W = \Omega(2^k)$  by first showing that the
expected contribution of all bids of value less than  $2^k$ 
to W is  $\Omega(2^k)$  and then showing that the contribution
of all bids of value  $2^k$  is  $\Omega(2^k)$ .
For  $i < k$ , the probability that a bid of value  $2^i$ 
wins is less than
i
i2k
i2k - 2i
 $\leq$ 
 $\leq$ 
i
T -2
T
k
The expected contribution of a winning bid is less
than
i
1[U+0001 control character] j
1
2 < 2^{i+1} .
i j=1
i
```

An Upper Bound

A natural question to ask is if the bound of Theorem 7.1 can be improved upon. We already know that no single-price auction can do better than F . In this section we prove that no truthful multi-price auction can have an expected revenue greater than F . This result contrasts that of Section 6. In that

Expanded PaperInterface specification

```
∀ (s : ℕ) (hs_two : 2 ≤ s),
  GHW01DigitalGoods.ProofBridge.weightedPairingRevenue (EconCSLib.Auction.ghwTightValue (s * s + 2) s) ≤
    6 / √(Real.logb 2 (2 ^ (s * s + 2))) *
    GHW01DigitalGoods.ProofBridge.ghwTightFixedPriceBenchmarkValue (s * s + 2) s hs_two
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem7_2_tightness

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: cited publication, lines 746-795

Verbatim source input

Now we show that in a truthful auction, probabilities of winning are monotone functions of bid values.

Lemma 8.1 Suppose in a truthful auction $b_i < b_j$.

Then $p_i \leq p_j$.

This all bids of value $2i$ contribute less than $2k+1/k$.

Summing over $i = 1, \dots, k-1$, we conclude that

the expected contribution of the corresponding bids

is less than $2 \cdot 2k$.

Next we consider the bids of value $2k$. The probability that such a bid wins is at least one and the expected revenue of a winning bid is at least

Proof. Suppose that i and j have the same utility equal to b_i . Since the auction is truthful, i 's gain is at least as big as j 's, thus

$p_i (b_i - c_i) \geq p_j (b_i - c_j)$.

Similarly, if both utilities are b_j , we get

```
k-1
s
s+1 k
1 [U+0001 control character] i
2k
k
2 +
2 =
.
2 <
k+s
k + s i=1
k+s
s
```

$p_j (b_j - c_j) \geq p_i (b_j - c_i)$.

Thus the total contribution of such bids is less than Adding the above inequalities and simplifying, we get

```
2k .
p b +p b ≥p b +p b .
i i
```

$j j$

$j i$

$i j$

Although the weighted pairing auction is not competitive in the worst case, it is only a factor of $\log h$

petitive in the worst case, it is only a factor of $\log h$

$p_j (b_j - b_i) \geq p_i (b_j - b_i)$.

away from being competitive, is quite different from

our random sampling auctions, and performs relatively well. Since $b_j > b_i$, we have $p_j \geq p_i$.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

For finite offer-price support, bidder values, prices, and bidder-specific offer laws satisfying the journal Definition 8.1

→ condition that $b_i \leq b_j$ and $x \leq b_i$ imply $\text{CDF}_i(x) \leq \text{CDF}_j(x)$, every strict inequality $b_i < b_j$ implies that bidder i 's

→ probability of receiving an offer at most b_i is at most bidder j 's probability of receiving an offer at most b_j .

Archival source anchor: cited publication, lines 746-795

Recorded basis Documented source correction 2026-08-02: retain archival SODA Lemma 8.1, replace the truthfulness-only route with the separately pinned journal Definition 8.1 monotone-offer target, and retain own-bid monotonicity as support; archival equivalence is not claimed.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} {Price : Type u_2} [inst : Fintype Price] (values : Agent → ℝ) (price : Price → ℝ)
  (offerLaw : Agent → PMF Price),
  (∀ (i j : Agent),
    values i ≤ values j →
      ∀ t ≤ values i,
        (EconCSLib.pmfProb (offerLaw i) fun p => price p ≤ t) ≤ EconCSLib.pmfProb (offerLaw j) fun p => price p ≤ t) →
  ∀ (i j : Agent),
    values i < values j →
      (EconCSLib.pmfProb (offerLaw i) fun p => price p ≤ values i) ≤
        EconCSLib.pmfProb (offerLaw j) fun p => price p ≤ values j
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_lemma8_1_monotone_allocation

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 14

Source locator: cited publication, lines 833-971

Verbatim source input

Theorem 8.2 For any truthful auction, $E[R] \leq F$.

$j=1$

$j=1$

$g_i \geq p_{i-1} (b_i - b_{i-1} + b_{i-1} - c_{i-1})$

$= p_n b_n +$

$= p_{i-1} (b_i - b_{i-1}) + p_{i-1} (b_{i-1} - c_{i-1})$

$= p_{i-1} (b_i - b_{i-1}) + g_{i-1}.$

$g_i \geq$

$p_j [b_j - (b_{j+1} - b_j)(n - j)] .$

$n-1$

[U+0001 control character]

$p_j [b_j (n - j + 1) - b_{j+1} (n - j)] .$

$j=1$

Regrouping

We can recursively expand g_{i-1} in the same way until we get to g_1 which is 0 because all bids are satisfied at value at least 1, and get

$i-1$

[U+0001 control character]

$n-1$

[U+0001 control character]

$E[R] \leq p_n b_n +$

$j=1$

(3) Now, let $V_j = b_j (n - j + 1)$. Intuitively, this is the revenue attained by using b_j as the sale price in fixed pricing. Note that $V_j \leq F$. Now let R_i be the total expected revenue from bidder i . That is

$n-1$

[U+0001 control character]

$E[R] \leq p_n V_n +$

$p_j (V_j - V_{j+1}) .$

$R_i = p_i c_i .$

$j=1$

$j=1$

$p_j (b_{j+1} - b_j) .$

We can rearrange equation (1) as $p_i c_i = p_i b_i - g_i$ and Rearrange this sum to sum over V_j instead of p_j and get for symmetry, define $p_0 = 0$.

$R_i = p_i b_i - g_i .$

$E[R] \leq$

Using equation (3) we get

$R_i \leq p_i b_i -$

$i-1$

[U+0001 control character]

$j=1$

n

[U+0001 control character]

$i=1$

$\leq$

n

[U+0001 control character]

$p_j (b_{j+1} - b_j) .$

$E[R] \leq F$

$i=1$

$p_i b_i -$

$i=1$

[U+F8EE private-use glyph]

[U+F8F0 private-use glyph]

$i-1$
 [U+0001 control character]
 $j=1$
 $(p_j - p_{j-1}) V_j$.
 n
 [U+0001 control character]
 $j=1$
 $(p_j - p_{j-1})$
 This sum telescopes to $p_n - p_0$, but since $p_0 = 0$, we have
 R_i
 n
 [U+0001 control character]
 $j=1$
 But, $V_j \leq F$ and by Lemma 8.1, $p_j - p_{j-1}$ is nonnegative.

Looking at the sum of the R_i 's, we see that the first term is mostly canceled by the summation term and we get a telescoping effect.
 $E[R] =$

n
 [U+0001 control character]
 $E[R] \leq p_n F \leq F$.
 [U+F8F9 private-use glyph]
 $p_j (b_{j+1} - b_j)$ [U+F8FB private-use glyph] .
 9

[form-feed in source extraction]
 9

Deterministic Auctions

optimal threshold auction on this input family has revenue of $R = F/h$.
 We have shown several randomized auctions with an
 Next we generalize this result.
 $\Omega(F)$ expected performance under the assumption that $ah \leq F$ and α is big enough. In this section we study deterministic auctions.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

For a finite randomized offer auction with nonnegative bid values and nonnegative prices, suppose that whenever $b_i \leq b_j$ and $x \leq b_i$, the probability that bidder i wins at a price at most x is at most the corresponding probability for bidder j . Then
 $\hookrightarrow$ its expected revenue on every bid vector is at most the optimal fixed-price revenue F of that vector.

Archival source anchor: cited publication, lines 833-833

Recorded basis Documented source correction 2026-07-26: replace the preliminary SODA Theorem 8.2 route with the separately pinned journal Definition 8.1/Theorem 8.2 target; archival equivalence is not claimed.

Expanded PaperInterface specification

$\forall \{ \text{Agent} : \text{Type } u_1 \} \{ \text{Price} : \text{Type } u_2 \} [\text{inst} : \text{Fintype Agent}] [\text{inst}_1 : \text{Nonempty Agent}] [\text{inst}_2 : \text{Fintype Price}]$
 $(\text{values} : \text{Agent} \rightarrow \mathbb{R}) (\text{price} : \text{Price} \rightarrow \mathbb{R}) (\text{offerLaw} : \text{Agent} \rightarrow \text{PMF Price}),$
 $(\forall (i : \text{Agent}), 0 \leq \text{values } i) \rightarrow$
 $(\forall (p : \text{Price}), 0 \leq \text{price } p) \rightarrow$
 $(\forall (i j : \text{Agent}),$
 $\text{values } i \leq \text{values } j \rightarrow$
 $\forall t \leq \text{values } i,$
 $(\text{EconCSLib.pmfProb } (\text{offerLaw } i) \text{ fun } p \Rightarrow \text{price } p \leq t) \leq$
 $\text{EconCSLib.pmfProb } (\text{offerLaw } j) \text{ fun } p \Rightarrow \text{price } p \leq t) \rightarrow$
 $\text{EconCSLib.Auction.paper_theorem8_2_raw_cdf_expected_revenue values price offerLaw} \leq$
 $\text{GHW01DigitalGoods.ProofBridge.fixedPriceBenchmark values}$

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem8_2_journal_revenue_upper_bound

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 15

Source locator: cited publication, lines 979-1090

Verbatim source input

truthful deterministic bid-independent auction has Theorem 9.1 For any truthful deterministic bidder $O(F/h)$ worst-case performance even for $ah \leq F$. independent auction and any constant α , there exists

Then we show that any deterministic truthful auction an input for which $R/F = O(1/h)$ and $ah \leq F$. is bid-independent. This implies an upper bound on worst-case performance for all truthful deterministic Proof. Let f be the (deterministic) function that defines the auction. Consider an input with n_h bids at value h and n_{ℓ} bids at value 1 . Restricted to such inputs, f is a function of n_h and n_{ℓ} . Note that we

9.1 Deterministic Optimal Threshold can assume, without loss of generality, that f takes on Auction only two values: 1 , in which case all bids are satisfied at the price of 1 , and h , in which case the high bids are satisfied at price h . Other values of f lead to bid-independent auction with $f = \text{opt}$, the optimal smaller revenues.

threshold function defined in Section 4. The only difWe wish to find an input family such that the ference between the deterministic optimal threshold bid-independent auction with function f has revenue $O(F/h)$. To obtain this family, we chose n_h and n_{ℓ} in that the former uses threshold $\text{opt}(B_i)$ for bidder i such a way that $f(n_h - 1, n_{\ell}) = 1$, $f(n_h, n_{\ell} - 1) = h$, and the latter uses $\text{opt}(B)$. Recall that B_i and B only differ in that b_i is not in B . From this, we

$R = n_h h + n_{\ell}$ and $F = n_h h + n_{\ell}$ so $R/F \leq 1/h$. Our goal now might also expect that for large n with suitable conis, given a deterministic auction would perform to within a constant fraction Consider the n_h, n_{ℓ} plane. For a fixed m look at of F . However, this is not the case as the following the line $n_h = k$ and $n_{\ell} = m - k$, and consider the line example illustrates.

The input that shows that the deterministic opti- segment connecting $(0, m)$ and $(m, 0)$. We need to mal threshold auction performance is $O(F/h)$ is one find a value of k with $k > \alpha$ where, when k increases with r bids at value h and $q = (h - 1)r - 1$ bids by one, f changes from 1 to h . Set $m = h^2 \alpha$. Assume that $f(\alpha, m - \alpha) = 1$. As at value 1 . On this input, the optimal single-price auction takes r high bids at price h for a revenue of we increase k from $k = \alpha$ the value of $f(k, m - k) = F = hr$. Note that the second-best threshold would must change from 1 to h because for $k = m$ we have take all n bids at price 1 for revenue of $hr - 1$. The $f(k, m - k) = f(m, 0) = h$. Thus it must be at some

*
*
*
*
*
*
deterministic optimal threshold auction takes all the k that $f(k, m - k) = h$ and $f(k - 1, m - k + 1) = 1$.

*
 high bids at value 1 and none of the low bids. This If we now choose $n_h = k$ and $n_{\ell} = m - k + 1$, we yields a revenue of $R = r$. None of the low bids are satisfy our criteria that $n_h > \alpha$ and that $f(n_h - 1, n_{\ell}) = 1$ and $f(n_h, n_{\ell} - 1) = h$. Thus, $R/F =$ a low bid from the optimal threshold auction does $O(1/h)$.
 not change the threshold). The high bids are taken
 Suppose now that our assumption that $f(\alpha, m - \alpha) = 1$ is false and instead it is h . Then for $n_h = k =$ at value 1 because removing one h causes the opti- $\alpha = 1$ is false and instead it is h . Then for $n_h = k =$ mal threshold to switch to 1. Thus, the deterministic α and $n_{\ell} = m - k + 1$ we have $F = m + 1 = h/2 \alpha + 1$
 10

[form-feed in source extraction]
 value b' would be better off bidding b'' and paying only v . Since the auction is truthful, it must be that $g(b') = v$.

and $R \leq h\alpha$ so
 $R/F \leq h/2\alpha$
 $\alpha + 1$
 $\leq h/2\alpha$
 2α

= $h/1$

Thus for any bid-independent auction with deterministic function f there exists an input family such that $R = O(F/h)$.

Expanded PaperInterface specification

```

∀ (rule : GHW01DigitalGoods.PaperBidIndependentBoundaryRule) {highValue : ℕ} {alpha : ℝ},
  2 ≤ highValue →
  0 < alpha →
  ∃ highCount lowCount,
    0 < highCount ∧
    GHW01DigitalGoods.twoValueBidIndependentBoundaryRevenue rule highValue highCount lowCount /
      EconCSLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount ≤
      1 / ↑highValue ∧
      ↑highValue * alpha ≤ EconCSLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount

```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem9_1_bid_independent_lower_bound

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 16

Source locator: cited publication, lines 1105-1128

Verbatim source input

Lemma 9.2 Any truthful deterministic auction is bid-independent.
Proof. Let $B_i = B \setminus \{i\}$ as before, and let B_{ix} be the set of bids with b_i replaced with value x . Let A be a truthful deterministic auction. Let $A(B)$ denote the result of running A on set B and let $A_i(B)$ denote the result for bidder i (∞ if i is rejected). Consider function g defined as $g(x) = A_i(B_{ix})$. Here g is clearly a function of A and B_i . We show that g is ∞ everywhere except for some region (v, ∞) , $[v, \infty)$, or $\emptyset$ where it has value v . This would imply that A is bid-independent. If $g(x) = \infty$ for all x then we have the empty interval, $\emptyset$. Otherwise, let $b = \inf\{x : g(x) < \infty\}$ and let $v = \min_x g(x)$. We now show that $b = v$ and for all $b' > b$, $g(b') = v$.
1. $v = b$
First we show that $v \leq b$. This is simple to see because $g(x)$ is defined so that either $g(x)$ is ∞ (and a bid value of x will lose) or $g(x) \leq x$ (and a bid of value x will win at price $g(x)$ which is necessarily at most x).
Now we show that $v \geq b$. Assume for a contradiction that there exists a b' such that $v < b' < b$. This auction is not truthful because a bidder with utility value b' would be better off bidding greater than b , win the auction and pay price v which is less than b' . Thus, $v = b$.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} (M : EconCSLib.Auction.DigitalGoodsAuction Agent),
  GHW01DigitalGoods.ProofBridge.truthful M →
    M.IndividuallyRational →
      M.NoPositiveTransfers →
        M.BinaryAllocation →
          ∀ (bids : Agent → ℝ) (i : Agent),
            EconCSLib.Auction.DeterministicOfferBidIndependent (EconCSLib.Auction.deterministicAuctionOffer M bids i)
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_lemma9_2_bid_independence

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 17

Source locator: cited publication, lines 1100-1104

Verbatim source input

Truthful Deterministic Auctions Theorem 9.3 For any truthful deterministic auction and any constant α , there exists an input for are Bid-Independent

The following lemma allows us to extend the result of Theorem 9.1 to arbitrary deterministic auctions.

[Next verbatim source excerpt]

which $R/F = O(1/h)$ and $\alpha h \leq F$.

Expanded PaperInterface specification

```
∀ {highValue : ℕ} {alpha : ℝ}
  (model : GHW01DigitalGoods.assumption_theorem9_3_primitive_set_of_bids_deterministic_source_model highValue),
  2 ≤ highValue →
  0 < alpha →
  ∃ highCount lowCount,
    0 < highCount ∧
    GHW01DigitalGoods.ProofBridge.revenue (model.auctionFamily highCount lowCount)
      (EconCSLib.Auction.twoValueBidProfile highValue highCount lowCount) /
      EconCSLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount ≤
      1 / ↑highValue ∧
      ↑highValue * alpha ≤ EconCSLib.Auction.twoValueFixedPriceBenchmark highValue highCount lowCount
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.result_theorem9_3_deterministic_lower_bound

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 18

Source locator: cited publication, lines 1599-1604

Verbatim source input

not selling all of the items. Note that the definitions of truthful and competitive auctions, which we stated for the unlimited supply case, also apply to the bounded supply case. We denote by F_k the revenue for optimal fixed pricing that sells at most k items, and it is this quantity that we wish to be competitive with.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : Nonempty Agent] (bids : Agent → ℝ) (capacity : ℕ),
  GHW01DigitalGoods.ProofBridge.source_boundedSupplyFk bids capacity =
    GHW01DigitalGoods.SourceDefinitions.boundedSupplyFk bids capacity
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_bounded_supply_fixed_price_benchmark_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 19

Source locator: cited publication, lines 1629-1631

Verbatim source input

to the optimal untruthful multi-price auction. Define T_k to be the revenue of the optimal multi-price auction restricted to only satisfying k bidders (i.e. the sum of the highest k bids) and F_k as above. Then

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (bids : Agent → ℝ) (capacity : ℕ)
(hcapacity : capacity ≤ Fintype.card Agent),
GHW01DigitalGoods.ProofBridge.source_boundedSupplyTk bids capacity hcapacity =
  GHW01DigitalGoods.SourceDefinitions.boundedSupplyTk bids capacity hcapacity
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_bounded_supply_top_k_total_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 20

Source locator: cited publication, lines 1680-1683

Verbatim source input

Let optk be the function that, given a set of bids,
For unlimited supply markets, our analysis as returns the optimal threshold that sells k items or sums that there is no cost for
 $\hookrightarrow$ producing the items
less. This function, on the entire set of bids, gives being auctioned. With the following modification we
the threshold to use for F_k . The single-price sampling can also accommodate non-zero marginal costs. If

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : Nonempty Agent] (bids : Agent → ℝ) (capacity : ℕ),
  GHW01DigitalGoods.ProofBridge.source_boundedSupplyOptK bids capacity =
    GHW01DigitalGoods.SourceDefinitions.boundedSupplyOptK bids capacity
```

Lean proof endpoint (built separately)

`GHW01DigitalGoods.PaperInterface.definition_bounded_supply_optimal_threshold_realizes_spec`

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 21

Source locator: cited publication, lines 1679-1690

Verbatim source input

We now show how to extend our optimal threshold simulations, we have argued that our auctions comsampling auctions to the k-item $\hookrightarrow$ bounded supply case. pare favorably to fixed pricing with market analysis.
Let opt_k be the function that, given a set of bids,
For unlimited supply markets, our analysis asreturns the optimal threshold that sells k items or sumes that there is no cost for $\hookrightarrow$ producing the items
less. This function, on the entire set of bids, gives being auctioned. With the following modification we the threshold to use for F_k . The single-price sampling can also accommodate non-zero marginal costs. If optimal threshold mechanisms can now be modified the marginal cost is v per item, then first subtract v to use threshold function $\text{opt}_m/(n-m)$ on sample of from each bid and reject all negative value bids. After size m . If this results in too many bids being satisfied, running the auction, add v back to the selling price arbitrarily (e.g. at random) reject bids until there are of all winning bids. If the marginal cost of producing only k left. One can show that with high probability, k items is a more complicated function of k , we can the number of bids rejected this way will be small modify the opt function used in the sampling optimal and that this auction is competitive.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : LinearOrder Agent] (sampleSize : ℕ) (hsample_pos : 0 < sampleSize)
(hsample_lt : sampleSize < Fintype.card Agent) (capacity : ℕ),
  GHW01DigitalGoods.ProofBridge.source_boundedSinglePriceSampling sampleSize hsample_pos hsample_lt capacity =
    GHW01DigitalGoods.SourceDefinitions.boundedSinglePriceFixedSizeOutcomeLaw sampleSize hsample_pos hsample_lt capacity
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.model_bounded_single_price_sampling_auction_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 22

Source locator: cited publication, lines 1692-1696

Verbatim source input

In the dual-price auction with the sample size of costs. In this case the opt function would, as above $m = n/2$, use $\text{opt}k/2$ so that about $k/2$ bids are selected from each of the sample and the non-sample. ratio of the sample size and the non-sample size so as Once can show that the resulting auction is also to correctly use marginal cost information. truthful and competitive.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : LinearOrder Agent] (sampleSize : ℕ) (hsample_pos : 0 < sampleSize)
(hhalf : Fintype.card Agent = sampleSize + sampleSize) (halfCapacity : ℕ),
GHW01DigitalGoods.ProofBridge.source_boundedDualPriceSampling sampleSize hsample_pos hhalf halfCapacity =
GHW01DigitalGoods.SourceDefinitions.boundedDualExactHalfOutcomeLaw sampleSize hsample_pos hhalf halfCapacity
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.model_bounded_dual_price_sampling_auction_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 23

Source locator: cited publication, lines 67-70

Verbatim source input

In this paper we study a class of truthful auctions for unlimited supply. We study both singleprice auctions, where every winning
↪ bidder pays the
same price, and multi-price auctions, where the prices
may differ. In addition to deterministic auctions, we
study randomized auctions that use randomization to

Expanded PaperInterface specification

```
(∀ {Agent : Type u_1} (outcome : GHW01DigitalGoods.ProofBridge.source_auctionOutcome Agent),  
  GHW01DigitalGoods.ProofBridge.source_singlePriceOutcome outcome =  
    GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.IsSinglePrice outcome) ∧  
(∀ {Agent : Type u_2} (auction : GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent),  
  GHW01DigitalGoods.ProofBridge.source_deterministicIsSinglePrice auction =  
    GHW01DigitalGoods.SourceDefinitions.DeterministicIsSinglePrice auction) ∧  
(∀ {Agent : Type u_3} (auction : GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent),  
  GHW01DigitalGoods.ProofBridge.source_randomizedIsSinglePrice auction =  
    GHW01DigitalGoods.SourceDefinitions.RandomizedIsSinglePrice auction) ∧  
(∀ (Agent : Type u_4),  
  GHW01DigitalGoods.ProofBridge.source_deterministicMultiPriceAuction Agent =  
    GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent) ∧  
  ∀ (Agent : Type u_5),  
    GHW01DigitalGoods.ProofBridge.source_randomizedMultiPriceAuction Agent =  
      GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_price_classes_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 24

Source locator: cited publication, lines 79-83

Verbatim source input

We view auctions as algorithms for deciding which input bids to fill, and at what price. As with any algorithm, one needs to address the issues of correctness, efficiency, and performance. In the context of this paper, an auction is correct if the auction is truthful and fills each winning bid at or below the bid value.

Expanded PaperInterface specification

```
(∀ {Agent : Type u_1} (auction : GHW01DigitalGoods.ProofBridge.source_deterministicAuction Agent),
  GHW01DigitalGoods.ProofBridge.source_deterministicCorrect auction =
    GHW01DigitalGoods.SourceDefinitions.DeterministicCorrect auction) ∧
  ∀ {Agent : Type u_2} (auction : GHW01DigitalGoods.ProofBridge.source_randomizedAuction Agent),
    GHW01DigitalGoods.ProofBridge.source_randomizedCorrect auction =
      GHW01DigitalGoods.SourceDefinitions.RandomizedCorrect auction
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_auction_correctness_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 25

Source locator: cited publication, lines 89-96

Verbatim source input

Consider the problem of selling a number of identical items to consumers who each want a single item and the items are available in unlimited supply. By unlimited supply we mean that either the seller has at least as many items as there are consumers, or that the seller can reproduce items on demand at negligible marginal cost. Of particular interest are digital items such as ↪ downloadable audio files and payper-view movies. With unlimited supply, consumer utilities, the maximum amounts that consumers are willing to pay for the item, are the sole factor determining sale prices. The seller's goal is to maximize their total revenue.

Expanded PaperInterface specification

```
(∀ (Agent : Type u_1),
  GHW01DigitalGoods.ProofBridge.source_unlimitedSupplyDeterministicAuction Agent =
    GHW01DigitalGoods.SourceDefinitions.UnlimitedSupplyDeterministicAuction Agent) ∧
∀ (Agent : Type u_2),
  GHW01DigitalGoods.ProofBridge.source_unlimitedSupplyRandomizedAuction Agent =
    GHW01DigitalGoods.SourceDefinitions.UnlimitedSupplyRandomizedAuction Agent
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.model_unlimited_supply_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 26

Source locator: cited publication, lines 97-103

Verbatim source input

One way to set prices for items in unlimited supply is to estimate consumer utility via market analysis and then set a fixed price. We refer to this method as fixed pricing. Pay-per-view movies are an example of fixed pricing for an unlimited supply market. With perfect knowledge of consumer utilities, optimal fixed pricing maximizes fixed-price revenue by selecting the optimal price at which to sell items. Fixed pricing generally cannot achieve this ideal due to the

Expanded PaperInterface specification

```
(∀ {Agent : Type u_1} [inst : Fintype Agent] (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent) (p : ℝ),
  GHW01DigitalGoods.ProofBridge.source_fixedPriceOutcome bids p =
    GHW01DigitalGoods.SourceDefinitions.fixedPriceOutcome bids p) ∧
(∀ {Agent : Type u_2} [inst : Fintype Agent] (p : ℝ),
  GHW01DigitalGoods.ProofBridge.source_fixedPriceAuction p =
    GHW01DigitalGoods.SourceDefinitions.fixedPriceAuction p) ∧
(∀ {Agent : Type u_3} [inst : Fintype Agent] (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent) (p : ℝ),
  GHW01DigitalGoods.ProofBridge.source_fixedPriceRevenue bids p =
    GHW01DigitalGoods.SourceDefinitions.fixedPriceRevenue bids p) ∧
(∀ {Agent : Type u_4} [inst : Fintype Agent] [inst_1 : Nonempty Agent]
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
  GHW01DigitalGoods.ProofBridge.source_optimalFixedPriceRevenue bids =
    GHW01DigitalGoods.SourceDefinitions.optimalFixedPriceRevenue bids) ∧
  ∀ {Agent : Type u_5} [inst : Fintype Agent] [inst_1 : Nonempty Agent]
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
  GHW01DigitalGoods.ProofBridge.source_optimalThresholdDefinition bids =
    GHW01DigitalGoods.SourceDefinitions.optimalThreshold bids
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_fixed_pricing_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 27

Source locator: cited publication, lines 128-130

Verbatim source input

To state our results more formally, we introduce the following notation. Let n denote the number of bidders. Without loss of generality, we assume that

Expanded PaperInterface specification

```
∀ (Agent : Type u_1) [inst : Fintype Agent],
  GHW01DigitalGoods.ProofBridge.source_bidderCount Agent = GHW01DigitalGoods.SourceDefinitions.bidderCount Agent
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_bidder_count_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 28

Source locator: cited publication, lines 130-132

Verbatim source input

bidders. Without loss of generality, we assume that the lowest bid is one and denote the highest bid by h . Let the total utility T be the sum of all bidders'

Expanded PaperInterface specification

```
(∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : Nonempty Agent]
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
  GHW01DigitalGoods.ProofBridge.source_lowestBid bids = GHW01DigitalGoods.SourceDefinitions.lowestBid bids) ∧
(∀ {Agent : Type u_2} [inst : Fintype Agent] [inst_1 : Nonempty Agent]
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
  GHW01DigitalGoods.ProofBridge.source_highestBid bids = GHW01DigitalGoods.SourceDefinitions.highestBid bids) ∧
  ∀ {Agent : Type u_3} [inst : Fintype Agent] [inst_1 : Nonempty Agent]
    (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
    GHW01DigitalGoods.ProofBridge.source_normalizedBidScale bids =
      GHW01DigitalGoods.SourceDefinitions.NormalizedBidScale bids
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_bid_extrema_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 29

Source locator: cited publication, lines 195-196

Verbatim source input

We consider auctions with n bidders, each bidder i having a utility value u_i and bidding b_i . We also assume that the bids are ordered so that $b_i \leq b_{i+1}$. In

Expanded PaperInterface specification

```
(∀ (Agent : Type u_1),
  GHW01DigitalGoods.ProofBridge.source_bidValues Agent = GHW01DigitalGoods.SourceDefinitions.BidValues Agent) ∧
(∀ (Agent : Type u_2),
  GHW01DigitalGoods.ProofBridge.source_bidProfile Agent = GHW01DigitalGoods.SourceDefinitions.BidProfile Agent) ∧
  ∀ (Agent : Type u_3),
    GHW01DigitalGoods.ProofBridge.source_auctionInput Agent = GHW01DigitalGoods.SourceDefinitions.AuctionInput Agent
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.model_bidder_values_and_bids_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 30

Source locator: cited publication, lines 196-199

Verbatim source input

having a utility value u_i and bidding b_i . We also assume that the bids are ordered so that $b_i \leq b_{i+1}$. In auctions where ties need to be broken, we can assume an arbitrary total order on the bid values that respects the partial order. That is, we can assume that the order given by the indices is strict. We assume that there is no collusion among the bidders.

Expanded PaperInterface specification

```
∀ {n : ℕ} (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile (Fin n)),  
  GHW01DigitalGoods.ProofBridge.source_sortedBidConvention bids = GHW01DigitalGoods.SourceDefinitions.SortedBids bids
```

Lean proof endpoint (built separately)

```
GHW01DigitalGoods.PaperInterface.model_sorted_bid_convention_realizes_spec
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 31

Source locator: cited publication, lines 200-203

Verbatim source input

Given a set of bids, the outcome of an auction is the subset of bids that are satisfied and a corresponding set of sale prices such that, for each winning bid b_i , the associated sale price is at most b_i . A deterministic auction mechanism maps sets of bids to auction

Expanded PaperInterface specification

```
(∀ (Agent : Type u_1),
  GHW01DigitalGoods.ProofBridge.source_auctionOutcome Agent =
    GHW01DigitalGoods.SourceDefinitions.AuctionOutcome Agent) ∧
  ∀ {Agent : Type u_2} (outcome : GHW01DigitalGoods.ProofBridge.source_auctionOutcome Agent)
    (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
    GHW01DigitalGoods.ProofBridge.source_outcomeFillsAtOrBelow outcome bids =
      GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.FillsAtOrBelow outcome bids
```

Lean proof endpoint (built separately)

```
GHW01DigitalGoods.PaperInterface.definition_auction_outcome_realizes_spec
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 32

Source locator: cited publication, lines 203-210

Verbatim source input

the associated sale price is at most b_i . A deterministic auction mechanism maps sets of bids to auction outcomes. A randomized auction mechanism maps

1 Some of our results hold under weaker assumptions.

2

[form-feed in source extraction]
sets of bids to probability distributions on auction

Expanded PaperInterface specification

```
(∀ (Agent : Type u_1),
  GHW01DigitalGoods.ProofBridge.source_deterministicAuction Agent =
    GHW01DigitalGoods.SourceDefinitions.DeterministicAuction Agent) ∧
  ∀ (Agent : Type u_2),
    GHW01DigitalGoods.ProofBridge.source_randomizedAuction Agent =
      GHW01DigitalGoods.SourceDefinitions.RandomizedAuction Agent
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_auction_mechanisms_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 33

Source locator: cited publication, lines 215-218

Verbatim source input

We say that an auction is truthful if bidding u_i is a dominant strategy for bidder i . More specifically, let a bidder's profit be the difference between the bidder's utility value and the price the bidder pays if they win the auction, or zero if they lose. Then an auction

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} (outcome : GHW01DigitalGoods.ProofBridge.source_auctionOutcome Agent)
  (values : GHW01DigitalGoods.ProofBridge.source_bidValues Agent) (i : Agent),
  GHW01DigitalGoods.ProofBridge.source_profitDefinition outcome values i =
    GHW01DigitalGoods.SourceDefinitions.AuctionOutcome.profit outcome values i
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_bidder_profit_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 34

Source locator: cited publication, lines 239-245

Verbatim source input

Analogous to on-line algorithm theory, we express auction performance relative to that for optimal untruthful auctions, as ratios R/T or R/F . However, we solve a maximization problem while on-line algorithms solve minimization problems. Thus, positive results, which are lower bounds on R/T or R/F , are expressed using “ Ω ”. Impossibility results, which are upper bounds on R/T or R/F for any auction in a certain class, are expressed using “ O ”.

Expanded PaperInterface specification

```
(∀ (auctionRevenue benchmark : ℝ),
  GHW01DigitalGoods.ProofBridge.source_performanceRatio auctionRevenue benchmark =
    GHW01DigitalGoods.SourceDefinitions.performanceRatio auctionRevenue benchmark) ∧
(∀ {Instance : Type u_1} (auctionRevenue benchmark : Instance → ℝ),
  GHW01DigitalGoods.ProofBridge.source_revenueBigOmega auctionRevenue benchmark =
    GHW01DigitalGoods.SourceDefinitions5.RevenueBigOmega auctionRevenue benchmark) ∧
  ∀ {Instance : Type u_2} (auctionRevenue benchmark : Instance → ℝ),
    GHW01DigitalGoods.ProofBridge.source_revenueBigO auctionRevenue benchmark =
      GHW01DigitalGoods.SourceDefinitions.RevenueBigO auctionRevenue benchmark
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_performance_convention_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 35

Source locator: cited publication, lines 257-265

Verbatim source input

can make the assumption that the optimal revenue F is significantly larger than h , the highest bid. That is, for some constant α , we assume that $\alpha h \leq F$. With this assumption, optimal fixed pricing sells at least α items. In some cases α is a fixed constant. In other cases, success probability approaches 1 as $\alpha \rightarrow \infty$. For some proofs, this assumption is stronger than what we need and we make the weaker assumption that $\alpha h \leq T$.

Expanded PaperInterface specification

```
(∀ (alpha highest fixedPrice : ℝ),
  GHW01DigitalGoods.ProofBridge.source_fixedPriceScaleCondition alpha highest fixedPrice =
    GHW01DigitalGoods.SourceDefinitions.fixedPriceScaleCondition alpha highest fixedPrice) ∧
(∀ (alpha highest total : ℝ),
  GHW01DigitalGoods.ProofBridge.source_totalValueScaleCondition alpha highest total =
    GHW01DigitalGoods.SourceDefinitions.totalValueScaleCondition alpha highest total) ∧
∀ (alpha : ℝ),
  GHW01DigitalGoods.ProofBridge.source_positiveAlpha alpha = GHW01DigitalGoods.SourceDefinitions.PositiveAlpha alpha
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_scale_conditions_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 36

Source locator: cited publication, lines 266-267

Verbatim source input

We say that an auction is competitive under certain assumptions if when the assumptions hold, the auction’s revenue is $\Omega(F)$, or equivalently $R/F = \Omega(1)$.

Expanded PaperInterface specification

```
(∀ {Instance : Type u_1} (admissible : Instance → Prop) (auctionRevenue benchmark : Instance → ℝ),
  GHW01DigitalGoods.ProofBridge.source_competitiveDefinition admissible auctionRevenue benchmark =
  GHW01DigitalGoods.SourceDefinitions.Competitive0n admissible auctionRevenue benchmark) ∧
  ∀ {Instance : Type u_2} (admissible : Instance → Prop) (auctionRevenue benchmark : Instance → ℝ),
    (∀ (x : Instance), admissible x → 0 < benchmark x) →
      (GHW01DigitalGoods.ProofBridge.source_competitiveDefinition admissible auctionRevenue benchmark ↔
        ∃ c,
          0 < c ∧
          ∀ (x : Instance),
            admissible x → c ≤ GHW01DigitalGoods.ProofBridge.source_performanceRatio (auctionRevenue x) (benchmark x))
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_competitive_auction_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 37

Source locator: cited publication, lines 284-287

Verbatim source input

The k-item Vickrey auction [18] was a starting place for our work. The k-item Vickrey auction is a single-price auction that sells k items to the k highest bidders at the price equal to the k + 1 highest bid (b_{n-k-1}). For generalizations of Vickrey auctions to

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (auction : GHW01DigitalGoods.ProofBridge.source_deterministicAuction Agent)
  (k : ℕ),
  GHW01DigitalGoods.ProofBridge.source_kItemVickreyAuction auction k =
    GHW01DigitalGoods.SourceDefinitions.IsKItemVickreyAuction auction k
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_k_item_vickrey_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 38

Source locator: cited publication, lines 289-291

Verbatim source input

In the unlimited supply case, taking $k = f(n)$ yields a truthful auction for any function f with $1 \leq f(n) < n$. This auction mechanism is not

Expanded PaperInterface specification

```
∀ (f : ℕ → ℕ),  
  GHW01DigitalGoods.ProofBridge.source_admissibleVickreySupplyRule f =  
    GHW01DigitalGoods.SourceDefinitions.AdmissibleVickreySupplyRule f
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_unlimited_vickrey_family_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 39

Source locator: cited publication, lines 317-335

Verbatim source input

To get a better understanding of how the optimal single-price auction works, we define the optimal threshold function, $\text{opt}(B)$. This function on a set of bids B returns the fixed price at which items should be sold to achieve revenue F . In the optimal single-price auction, all bid values that are at least $\text{opt}(B)$ will be satisfied at price $\text{opt}(B)$. All other bids will be rejected. That is,

5

Generalized Truthful Auction Mechanisms

By making observations about auctions that encourage utility value bids, in particular the Vickrey auction, we can design general $\hookrightarrow$ auction mechanisms. We present two general auction mechanisms that facilitate the design of truthful auctions. The first, the bid-independent auction $\hookrightarrow$ mechanism, is based on the observation that the price that a bid is satisfied at must be independent from that bid's actual value. The second, the random sampling auction mechanism, is based on the observation that rejected bids can be used to set prices for bids that are to be satisfied.

$\text{opt}(B) = \text{argmax}_{b_i \in B} b_i \cdot (n - i + 1).$

Expanded PaperInterface specification

```
(∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : Nonempty Agent]
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent) (p : ℝ),
  GHW01DigitalGoods.ProofBridge.source_isOptimalThreshold bids p =
    GHW01DigitalGoods.SourceDefinitions.IsOptimalThreshold bids p) ∧
(∀ {Agent : Type u_2} [inst : Fintype Agent] [inst_1 : Nonempty Agent]
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
  GHW01DigitalGoods.ProofBridge.source_optimalThresholdDefinition bids =
    GHW01DigitalGoods.SourceDefinitions.optimalThreshold bids) ∧
(∀ {Agent : Type u_3} [inst : Fintype Agent] [inst_1 : Nonempty Agent]
  (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),
  GHW01DigitalGoods.ProofBridge.source_optimalSinglePriceOutcome bids =
    GHW01DigitalGoods.SourceDefinitions.optimalSinglePriceOutcome bids) ∧
∀ {Agent : Type u_4} [inst : Fintype Agent] [inst_1 : Nonempty Agent],
  GHW01DigitalGoods.ProofBridge.source_optimalSinglePriceAuction =
    GHW01DigitalGoods.SourceDefinitions.optimalSinglePriceAuction
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_optimal_threshold_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 40

Source locator: cited publication, lines 351-357

Verbatim source input

Let B be the set of all bids and let B_i be the set of bids without bidder i 's bid. A bid-independent auction uses a predetermined function, f , from sets of bid values to prices. The auction works as follows. Bidder i wins the auction at price $f(B_i)$ if $b_i \geq f(B_i)$.
2
Otherwise, the bidder loses.

Expanded PaperInterface specification

```
GHW01DigitalGoods.ProofBridge.source_thresholdOffer = GHW01DigitalGoods.SourceDefinitions.ThresholdOffer ∧  
  (∀ (offer : GHW01DigitalGoods.ProofBridge.source_thresholdOffer) (bid : ℝ),  
    GHW01DigitalGoods.ProofBridge.source_acceptsThreshold offer bid =  
      GHW01DigitalGoods.SourceDefinitions.accepts offer bid) ∧  
  (∀ {Agent : Type u_1} [inst : Fintype Agent] (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent)  
    (i : Agent),  
    GHW01DigitalGoods.ProofBridge.source_otherBids bids i = GHW01DigitalGoods.SourceDefinitions.otherBids bids i) ∧  
  GHW01DigitalGoods.ProofBridge.source_bidIndependentRule = GHW01DigitalGoods.SourceDefinitions.BidIndependentRule ∧  
  (∀ {Agent : Type u_2} [inst : Fintype Agent] (rule : GHW01DigitalGoods.ProofBridge.source_bidIndependentRule)  
    (bids : GHW01DigitalGoods.ProofBridge.source_bidProfile Agent),  
    GHW01DigitalGoods.ProofBridge.source_bidIndependentOutcome rule bids =  
      GHW01DigitalGoods.SourceDefinitions.bidIndependentOutcome rule bids) ∧  
  (∀ {Agent : Type u_3} [inst : Fintype Agent] (rule : GHW01DigitalGoods.ProofBridge.source_bidIndependentRule),  
    GHW01DigitalGoods.ProofBridge.source_bidIndependentAuction rule =  
      GHW01DigitalGoods.SourceDefinitions.bidIndependentAuction rule
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_bid_independent_auction_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 41

Source locator: cited publication, lines 372-374

Verbatim source input

2 For maximum generality, we allow f to also return whether to use $\geq$ or $>$ in this comparison. For the purpose of maximizing revenue we can allways just use $\geq$; however, to prove Lemma 9.2 we need the full generality.

Expanded PaperInterface specification

`GHW01DigitalGoods.ProofBridge.source_thresholdBoundaryConvention = GHW01DigitalGoods.SourceDefinitions.ThresholdBoundary`

Lean proof endpoint (built separately)

`GHW01DigitalGoods.PaperInterface.definition_threshold_boundary_convention_realizes_spec`

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 42

Source locator: cited publication, lines 410-416

Verbatim source input

Another general truthful auction mechanism is based on randomized sampling. We select a subset B' of B at random, independent of the bid values. Let m be the size of B' . We then compute a function on these sampled bids, $f(B')$, and use this as a threshold value for the $n-m$ bids in the non-sample, $B \setminus B'$. Note that this auction mechanism is inherently single-price.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (plan : GHW01DigitalGoods.ProofBridge.source_randomSamplingPlan Agent),
  GHW01DigitalGoods.ProofBridge.source_randomSamplingAuction plan =
    GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.directionalOutcomeLaw plan
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_random_sampling_auction_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 43

Source locator: cited publication, lines 417-422

Verbatim source input

If a multi-price auction is acceptable, then this auction can be modified to be dual-price with $m \approx n/2$, by using $f(B')$ to compute the threshold to use for bids in $B \setminus B'$ and $f(B \setminus B')$ as the threshold to use for bids in B' . This is a good way to avoid revenue loss due to the rejected bids in the sample; however, it is at the expense of making the auction dual-price.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (plan : GHW01DigitalGoods.ProofBridge.source_randomSamplingPlan Agent),
  GHW01DigitalGoods.ProofBridge.source_dualPriceRandomSamplingAuction plan =
    GHW01DigitalGoods.SourceDefinitions.RandomSamplingPlan.dualPriceOutcomeLaw plan
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_dual_price_sampling_auction_realizes_spec

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 44

Source locator: cited publication, lines 455-465

Verbatim source input

The random sampling optimal threshold auction takes $f = \text{opt}$, the optimal threshold function, in the random sampling auction. Intuitively we use the sample to get an idea for a good threshold value, then we apply that threshold to the remaining bids. The auction samples $m = n/2$ bids at random, computes opt of this sample, and uses this value as a threshold for the non-sample, accepting all bids above this threshold at the threshold value. In this section we show that the expected revenue of this auction is within a constant factor of F , assuming F/h is not too small.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (sampleSize : ℕ) (hsample_pos : 0 < sampleSize)
(hhalf : Fintype.card Agent = sampleSize + sampleSize)
(boundary : GHW01DigitalGoods.ProofBridge.source_thresholdBoundaryConvention),
GHW01DigitalGoods.ProofBridge.source_exactHalfOptimalThresholdAuction sampleSize hsample_pos hhalf boundary =
  GHW01DigitalGoods.SourceDefinitions.exactHalfOptimalThresholdOutcomeLaw sampleSize hsample_pos hhalf boundary
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_exact_half_optimal_threshold_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 45

Source locator: cited publication, lines 466-469

Verbatim source input

For the purpose of simplifying our analysis, we will be analyzing a different method of sampling, one that selects a bid to be in the sample independently at random with probability 1/2. This method of sampling is simpler to analyze, and it does worse than the for5

Expanded PaperInterface specification

```
∀ (Agent : Type u_1),
  GHW01DigitalGoods.ProofBridge.source_independentHalfSamplingAnalysisLaw Agent =
    GHW01DigitalGoods.SourceDefinitions.independentHalfSamplingAnalysisLaw Agent
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_independent_half_analysis_model_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 46

Source locator: cited publication, lines 495-508

Verbatim source input

Let k_s be the number of bids satisfied by optimal single-price or dual-price. In this section we describe fixed pricing on the sample (i.e. the number of bids a multi-price auction. The weighted pairing auction in the sample that are at least $\text{opt}(B')$) and let k_n we present is in the bid-independent class with the be the number of bids in the non-sample that are at function f defined as follows:
least $\text{opt}(B')$. If $F' \geq F/3$ then $k_s \geq \alpha/3$.
 $f(B) = b \in B$ w.p. [U+0002 control character] b
Now, assuming that $k_s \geq \alpha/3$, we show that the
 $b \in B$
probability that $k_n < k_s/2$ is small. Note that $k_n <$
 $k_s/2$ implies that among the top $(3/2)k_s$ bids, at least Thus, to determine if bidder i wins the auction and
 k_s are in the sample. Note that the sample and the at what price, pick a bid $b \in B_i$ with probability pronon-sample are symmetric:
 $\hookrightarrow$ taking a random subset portional to the value of b , i.e., $b/(T - b_i)$. This pairs
containing half of the elements in A is equivalent to b_i with b . If $b \leq b_i$, bidder i wins at cost b , othertaking a complement of
 $\hookrightarrow$ a random subset of half of wise i loses. Note that the result of this selection for
the elements. We apply Lemma 6.1 with S being i does not affect the auction outcome for the bidder
the non-sample, B being the top $i = (3/2)k_s$ bids, who bid b .

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (bids : Agent → ℝ) (hbid_ge_one : ∀ (i : Agent), 1 ≤ bids i)
(hdistinct : ∀ (i : Agent), ∃ j, j ≠ i),
GHW01DigitalGoods.ProofBridge.source_weightedPairingAuction bids hbid_ge_one hdistinct =
GHW01DigitalGoods.SourceDefinitions.weightedPairingOutcomeLaw bids --
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_weighted_pairing_auction_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 47

Source locator: cited publication, lines 991-1013

Verbatim source input

The deterministic optimal threshold auction is the
are satisfied at price h . Other values of f lead to
bid-independent auction with $f = \text{opt}$, the optimal
smaller revenues.
threshold function defined in Section 4. The only difWe wish to find an input family such that the
ference between the deterministic optimal threshold
bid-independent
auction with function f has revenue
auction and the optimal fixed pricing mechanism is
 $O(F$
 $/h)$.
To
obtain
this family, we chose n_h and n_{ell} in
that the former uses threshold $\text{opt}(B_i)$ for bidder i
such
a
way
that
 f
(n
 $h - 1, n_{ell}$) = 1, $f(n_h, n_{ell} - 1) = h$,
and the latter uses $\text{opt}(B)$. Recall that B_i and B

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (hcard : 2 ≤ Fintype.card Agent),
  GHW01DigitalGoods.ProofBridge.source_deterministicOptimalThresholdAuction hcard =
    GHW01DigitalGoods.SourceDefinitions.deterministicOptimalThresholdAuction hcard
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_deterministic_optimal_threshold_auction_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 48

Source locator: cited publication, lines 1595-1600

Verbatim source input

Up to this point, we have studied the unlimited supply case which is motivated by the digital goods market where the cost of $\hookrightarrow$ making a copy of an item is negligible. In this section we consider the case where the number of items available for sale is bounded. This case is typical of physical goods markets. We denote the number of items available by k . Here again, the seller wishes to maximize their revenue, possibly not selling all of the items. Note that the definitions of truthful and competitive auctions, which we stated for the unlimited supply case, also apply to the

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] (auction : EconCSLib.Auction.DigitalGoodsAuction Agent) (capacity : ℕ),
  GHW01DigitalGoods.ProofBridge.source_boundedSupplyModel auction capacity =
    GHW01DigitalGoods.SourceDefinitions.IsKItemBoundedSupplyAuction auction capacity
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.model_bounded_supply_capacity_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 49

Source locator: cited publication, lines 1623-1625

Verbatim source input

4 Formally, items are in scarce supply if we have k items and all of the sample and the loss due to having a non- the optimal fixed pricing with k or $k + 1$ items would sell all of the items.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} [inst : Fintype Agent] [inst_1 : Nonempty Agent] (bids : Agent → ℝ) (capacity : ℕ),
  GHW01DigitalGoods.ProofBridge.source_scarceSupply bids capacity =
    GHW01DigitalGoods.SourceDefinitions.scarceSupply bids capacity
```

Lean proof endpoint (built separately)

GHW01DigitalGoods.PaperInterface.definition_scarce_supply_realizes_spec

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation